

# Cloud Computing Basics Cheat Sheet

16 tables · 142 concepts · Updated 2026-09-27 · [cheatgrid.com](https://cheatgrid.com)

**Cloud computing** is the on-demand delivery of IT resources over the Internet with **pay-as-you-go pricing**. Rather than buying and maintaining physical data centers, organizations provision **compute, storage, databases, networking and other services** instantly from providers like AWS, Microsoft Azure and Google Cloud Platform. The mental model that matters: cloud shifts IT from a capital expenditure (buying servers) to an operational expenditure (renting capacity), trading upfront hardware for **agility, scalability and cost efficiency** while teams spend their time on the product instead of infrastructure management.

## CONTENTS

|                                             |    |
|---------------------------------------------|----|
| Essential Characteristics (NIST SP 800-145) | 1  |
| Service Models                              | 2  |
| Deployment Models                           | 2  |
| Core Benefits                               | 3  |
| Global Infrastructure                       | 4  |
| Computing Resources                         | 4  |
| Storage Services                            | 5  |
| Networking Concepts                         | 5  |
| Database Services                           | 6  |
| Security and Compliance                     | 7  |
| Pricing and Cloud Economics                 | 8  |
| Resource Organization and Governance        | 9  |
| Management and Monitoring                   | 10 |
| Development and DevOps                      | 10 |
| Cloud Adoption and Migration                | 11 |
| Well-Architected Framework Pillars          | 12 |
| References                                  | 12 |

TABLE 1

## Essential Characteristics (NIST SP 800-145)

These five traits are NIST's official definition of what makes something genuinely cloud rather than just hosting, and they show up constantly on certification exams. Together they describe a system a consumer can provision without human interaction, reach from any device, that pools resources across many tenants, scales elastically with demand, and meters usage so consumers pay only for what they use.

| Characteristic                | Example                                                 | Description                                                                                                                                                                                     |
|-------------------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>On-Demand Self-Service</b> | Launching a VM via AWS console without calling support  | A consumer can <b>provision resources automatically</b> whenever needed, without a phone call or ticket to the provider                                                                         |
| <b>Broad Network Access</b>   | Accessing cloud applications from laptop, phone, tablet | Resources are reachable <b>over the network through standard mechanisms</b> , from any client device: phone, tablet, laptop, or workstation                                                     |
| <b>Resource Pooling</b>       | Multiple tenants sharing same physical servers          | The provider's compute, storage, and network resources are pooled and <b>dynamically assigned across many tenants</b> (multi-tenancy), usually with no control over the exact physical location |
| <b>Rapid Elasticity</b>       | Auto-scaling web servers during traffic spike           | Capacity can be <b>provisioned and released quickly</b> , often automatically, to scale with demand and appear unlimited to the consumer                                                        |
| <b>Measured Service</b>       | Pay-per-GB storage, pay-per-hour compute                | Usage is <b>metered, monitored, and reported</b> at a level suited to the service, giving both provider and consumer visibility into consumption                                                |

TABLE 2

## Service Models

Service models answer one question: how much of the stack does the provider manage versus you? The classic trio, IaaS, PaaS, and SaaS, runs from renting raw virtual machines to consuming finished applications in a browser, trading control for convenience at each step: IaaS suits custom or legacy workloads, PaaS speeds up app development, and SaaS fits ready-made business tools. The newer as-a-service rows extend the same idea to functions, databases, AI, containers, and more, so you can outsource exactly the layer that isn't your core problem.

| Model                                            | Example                                                         | Description                                                                                                                                                                                                          |
|--------------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Infrastructure as a Service (IaaS)</b>        | AWS EC2, Azure Virtual Machines, Google Compute Engine          | Provides <b>virtualized computing resources</b> over the internet: you manage the OS, runtime, middleware, and applications while the provider manages the physical infrastructure, networking, and storage hardware |
| <b>Platform as a Service (PaaS)</b>              | AWS Elastic Beanstalk, Azure App Service, Google App Engine     | Offers a complete <b>development and deployment environment</b> in the cloud: it handles infrastructure, OS, middleware, and runtime so developers focus solely on code and data                                     |
| <b>Software as a Service (SaaS)</b>              | Microsoft 365, Salesforce, Google Workspace, Dropbox            | Delivers <b>fully managed applications</b> over the internet: users access them via a browser with no infrastructure to manage, automatic updates, and subscription pricing                                          |
| <b>Function as a Service (FaaS / Serverless)</b> | AWS Lambda, Azure Functions, Google Cloud Functions             | Enables <b>event-driven code execution</b> without managing servers: pay only for actual compute time in milliseconds, with automatic scaling from zero to thousands of concurrent executions                        |
| <b>Database as a Service (DBaaS)</b>             | Amazon RDS, Azure SQL Database, Google Cloud SQL                | Provides <b>managed database services</b> with automated backups, patching, and scaling: eliminates database administration overhead while maintaining high availability                                             |
| <b>AI as a Service (AIaaS)</b>                   | Amazon SageMaker, Google Vertex AI, Azure AI Services           | Provides <b>managed AI and ML platforms</b> : pre-trained models, training infrastructure, and inference APIs without building ML pipelines from scratch                                                             |
| <b>Backend as a Service (BaaS)</b>               | Firebase, AWS Amplify, Azure Mobile Apps                        | Delivers <b>ready-to-use backend functionality</b> like authentication, databases, and APIs: primarily for mobile and web applications with minimal backend coding                                                   |
| <b>Container as a Service (CaaS)</b>             | Amazon ECS, Azure Container Instances, Google Kubernetes Engine | Offers <b>container orchestration and management</b> : deploy, scale, and manage containerized applications without managing the underlying infrastructure                                                           |
| <b>Network as a Service (NaaS)</b>               | Cisco Meraki, Megaport, Cloudflare Magic WAN                    | Delivers <b>virtualized networking services</b> over the cloud: replaces traditional hardware-centric networking with on-demand, software-defined connectivity                                                       |
| <b>Disaster Recovery as a Service (DRaaS)</b>    | Zerto, Azure Site Recovery, AWS Elastic Disaster Recovery       | Provides <b>cloud-based disaster recovery</b> : replicates and hosts physical or virtual servers to provide failover during a natural or man-made disaster                                                           |

TABLE 3

## Deployment Models

Where the infrastructure lives and who shares it is a separate decision from which service model you pick. Public cloud means shared, provider-owned infrastructure at maximum scale; private cloud is dedicated to one organization for tighter control; and hybrid and multi-cloud blend approaches to balance cost, compliance, and vendor lock-in. The sovereign and community rows reflect the growing pressure to keep data inside specific legal borders.

| Model               | Example                                     | Description                                                                                                                                                                                                                                                                              |
|---------------------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Public Cloud</b> | AWS, Microsoft Azure, Google Cloud Platform | <ul style="list-style-type: none"><li>• <b>Shared, provider-owned infrastructure</b> delivered over the internet to many customers (multi-tenancy).</li><li>• Maximum scalability and cost efficiency</li><li>• Best fit for public-facing apps, dev/test, and variable demand</li></ul> |

| Model                  | Example                                             | Description                                                                                                                                                                                                                                                                                                                   |
|------------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Private Cloud</b>   | VMware Cloud, OpenStack on-premises, Azure Stack    | <ul style="list-style-type: none"> <li>• <b>Dedicated infrastructure for one organization</b>, hosted on-premises or by a third party.</li> <li>• Greater control, security, and compliance</li> <li>• Best fit for regulated or steady, predictable workloads</li> </ul>                                                     |
| <b>Hybrid Cloud</b>    | AWS Outposts, Azure Arc, Google Anthos              | <ul style="list-style-type: none"> <li>• <b>Combines public and private clouds</b> with orchestration between them.</li> <li>• Moves workloads between environments for cost, compliance, or capacity</li> <li>• Best fit for cloud bursting and gradual cloud migration</li> </ul>                                           |
| <b>Multi-Cloud</b>     | Using AWS + Azure + GCP simultaneously              | <ul style="list-style-type: none"> <li>• <b>Uses several cloud providers at once</b>, rather than committing to one.</li> <li>• Avoids vendor lock-in and adds redundancy</li> <li>• Best fit for best-of-breed service selection, at the cost of more management overhead</li> </ul>                                         |
| <b>Community Cloud</b> | AWS GovCloud, Azure Government                      | <ul style="list-style-type: none"> <li>• <b>Shared infrastructure for organizations with common concerns</b>, such as mission, security, or compliance needs.</li> <li>• Owned or operated by one or more community members, or a third party</li> <li>• Best fit for government, healthcare, or finance consortia</li> </ul> |
| <b>Sovereign Cloud</b> | T-Systems Sovereign Cloud, Google Distributed Cloud | <ul style="list-style-type: none"> <li>• <b>Keeps data and operations inside national borders</b> under local jurisdiction and oversight.</li> <li>• Meets government data-residency and compliance mandates</li> <li>• Best fit for defense, public sector, and other regulated industries</li> </ul>                        |

TABLE 4

## Core Benefits

The concrete payoffs that pull organizations off their own data centers and onto the cloud. The headline is the shift from capital expense to operational expense, but the rest of the table matters just as much: scalability and elasticity to match capacity to demand, agility to ship faster, and the global reach, high availability, and security you'd struggle to build alone. A handy distinction to hold onto is scalability (the ability to grow) versus elasticity (growing and shrinking automatically).

| Benefit                                      | Example                                                                   | Description                                                                                                                                                                                                                                                         |
|----------------------------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cost Savings (CapEx to OpEx)</b>          | Trading \$100K data center investment for \$1K/month cloud bill           | Eliminates <b>capital expenses</b> for hardware and data centers, converts to variable operational expenses with pay-only-for-what-you-use pricing and economies of scale                                                                                           |
| <b>Scalability</b>                           | Adding 100 servers during holiday shopping season                         | Ability to <b>increase or decrease resources</b> to handle workloads: scale vertically (bigger machines) or horizontally (more machines) based on demand                                                                                                            |
| <b>Elasticity</b>                            | Auto-scaling from 2 to 200 instances and back to 2                        | <b>Automatic scaling</b> of resources based on current demand: scales both up and down dynamically, ensuring optimal resource utilization and cost efficiency                                                                                                       |
| <b>Agility</b>                               | Deploying new environment in minutes vs weeks                             | <b>Faster time-to-market</b> for new capabilities: provision resources in minutes, experiment rapidly, and iterate quickly without long procurement cycles                                                                                                          |
| <b>Global Reach</b>                          | Deploying an application across multiple AWS Regions in minutes           | Deploy applications in <b>multiple geographic locations worldwide</b> , putting them closer to end users and reducing latency                                                                                                                                       |
| <b>High Availability</b>                     | 99.95% monthly uptime SLA backed by the provider                          | Keeps a service running through disruptions or failures, backed by the provider's <b>service-level agreements (SLAs)</b> that guarantee uptime                                                                                                                      |
| <b>Security and Governance</b>               | Provider-managed DDoS protection, automatic patching, compliance auditing | <ul style="list-style-type: none"> <li>• Providers absorb large-scale threats such as <b>DDoS attacks</b></li> <li>• Templates, automatic patching, and auditing keep resources compliant and well governed</li> </ul>                                              |
| <b>Predictability (Performance and Cost)</b> | Using a cloud pricing calculator to forecast next month's spend           | <ul style="list-style-type: none"> <li>• <b>Performance predictability</b>: autoscaling and load balancing keep response times consistent under load</li> <li>• <b>Cost predictability</b>: real-time usage tracking and analytics forecast future spend</li> </ul> |

| Benefit              | Example                                                                                 | Description                                                                                                                                                                                                                                      |
|----------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Manageability</b> | Deploying resources from a template, then monitoring and scaling them via CLI or portal | <ul style="list-style-type: none"><li>• <b>Management of</b> the cloud: templated deployment, automatic scaling, and health monitoring</li><li>• <b>Management in</b> the cloud: control resources via portal, CLI, API, or PowerShell</li></ul> |

TABLE 5

## Global Infrastructure

Cloud providers organize their physical footprint into a nested geography that you'll deploy into deliberately. Regions are broad geographic areas built from one or more data centers, availability zones group those data centers so you can survive a single failure, and edge and local zones push compute closer to users for lower latency. Data sovereignty is the legal overlay: the rules dictating which region your data is even allowed to sit in.

| Concept                              | Example                                                                                                                  | Description                                                                                                                                                                                                                                     |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Region</b>                        | <code>us-east-1</code> (N. Virginia), <code>eu-west-1</code> (Ireland)                                                   | A <b>geographic area</b> with multiple isolated data center clusters, chosen for latency, compliance, and data residency requirements                                                                                                           |
| <b>Availability Zone (AZ)</b>        | <code>us-east-1a</code> , <code>us-east-1b</code> , <code>us-east-1c</code>                                              | <b>Isolated data center(s)</b> within a region, connected via low-latency links so a failure in one AZ does not affect others                                                                                                                   |
| <b>Data Center</b>                   | An AZ (e.g. <code>us-east-1a</code> ) can span several data center buildings sharing power and network                   | <ul style="list-style-type: none"><li>• The <b>physical building</b> with its own power, cooling, and networking</li><li>• one or more make up a single Availability Zone</li></ul>                                                             |
| <b>Edge Location</b>                 | CloudFront PoP in Tokyo caching static content                                                                           | <b>CDN endpoint</b> closer to end users, caching content at 400+ points of presence worldwide for lower latency content delivery                                                                                                                |
| <b>Local Zone</b>                    | AWS Local Zone in Los Angeles for low-latency gaming                                                                     | <b>Extension of a region</b> placed in a metro area, providing single-digit millisecond latency for latency-sensitive applications                                                                                                              |
| <b>Region Pair</b>                   | <code>West US</code> paired with <code>East US</code> , <code>Canada Central</code> paired with <code>Canada East</code> | <ul style="list-style-type: none"><li>• Two regions in the same geography that a service <b>replicates between</b> for geo-redundancy and disaster recovery</li><li>• planned updates are staggered so both are never patched at once</li></ul> |
| <b>Government / Sovereign Region</b> | AWS GovCloud (US-East, US-West) for regulated U.S. government workloads                                                  | <b>Isolated region</b> restricted to vetted users, operated to meet elevated compliance needs such as FedRAMP or handling Controlled Unclassified Information                                                                                   |
| <b>Data Sovereignty</b>              | GDPR requiring EU citizen data stays in EU                                                                               | Legal requirement that data is <b>subject to the laws</b> of the country where it is physically located, regardless of who owns or manages it                                                                                                   |

TABLE 6

## Computing Resources

These are the ways you actually run code in the cloud, arranged from heaviest to lightest. Virtual machines give you a full OS to control, sharing physical hardware through a hypervisor, while containers package just your app and its dependencies for faster, denser deployment. Bare metal and GPU instances handle performance-critical or accelerated workloads that need direct hardware access.

| Type                    | Example                                                               | Description                                                                                                                                                                                                                                        |
|-------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Virtual Machines</b> | <code>az vm create -g MyResourceGroup -n MyVm --image MyImage</code>  | <b>Virtualized servers</b> running a complete operating system, with full control over CPU, memory, and storage configuration                                                                                                                      |
| <b>Containers</b>       | Docker containers running on Amazon ECS or Kubernetes                 | <b>Lightweight virtualization</b> packaging application and dependencies, sharing the OS kernel for faster startup and higher density than VMs                                                                                                     |
| <b>Hypervisor</b>       | VMware ESXi and KVM (Type 1, bare metal), VirtualBox (Type 2, hosted) | <ul style="list-style-type: none"><li>• Software <b>layer that creates and runs VMs</b>, allocating physical hardware to each guest</li><li>• <b>Type 1</b> installs directly on hardware, <b>Type 2</b> runs as an app inside a host OS</li></ul> |

| Type                 | Example                                           | Description                                                                                                                                                                    |
|----------------------|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bare Metal Instances | AWS i3.metal, Azure Bare Metal, GCP C3 bare metal | <b>Physical servers</b> without a virtualization layer, providing maximum performance for workloads sensitive to virtualization overhead or requiring specific hardware access |
| GPU Instances        | NVIDIA H100 instances for LLM training            | Specialized compute with <b>graphics processing units</b> , accelerating machine learning, scientific computing, graphics rendering, and video encoding workloads              |

TABLE 7  
Storage Services

Cloud storage isn't one thing: the type you choose depends on how your data is accessed. Object storage holds unstructured files like media and backups at near-infinite scale, block storage gives VMs fast, low-latency volumes for databases, and file storage offers shared network filesystems multiple machines can mount at once. Access tiers and redundancy settings then decide how cheaply and how durably that data is kept.

| Type                             | Example                                                                      | Description                                                                                                                                                                                                                 |
|----------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Object Storage                   | Amazon S3, Azure Blob Storage, Google Cloud Storage                          | Stores data as <b>discrete objects with metadata</b> , highly scalable and durable (99.999999999% durability), accessed via HTTP APIs, ideal for unstructured data like images, videos, and backups                         |
| Block Storage                    | Amazon EBS, Azure Managed Disks, GCP Persistent Disk                         | Provides <b>raw storage volumes</b> attached to VMs, giving low-latency persistent storage for databases and applications requiring direct block-level access                                                               |
| File Storage                     | Amazon EFS, Azure Files, Google Filestore                                    | Offers <b>network-attached file systems</b> , supporting concurrent access from multiple instances using standard file protocols like NFS or SMB                                                                            |
| Storage Tiers                    | Hot (accessed daily), Cool (30+ days), Cold (90+ days), Archive (180+ days)  | <b>Access tiers</b> for the same account or bucket, priced by how often data is read: a hot tier costs more to store but less to access, cold and archive tiers reverse that trade                                          |
| Archive Storage                  | Amazon S3 Glacier, Azure Archive Storage                                     | <b>Long-term cold storage</b> at the lowest cost, optimized for data accessed rarely, with retrieval times from minutes to hours                                                                                            |
| Storage Redundancy / Replication | LRS (single datacenter), ZRS (3+ zones), GRS (replicated to a paired region) | <ul style="list-style-type: none"><li>• Copies data across datacenters, zones, or regions to protect against hardware or regional failure</li><li>• more redundancy costs more but survives a bigger blast radius</li></ul> |
| Lifecycle Policies               | Transition blobs to cool after 30 days, delete after 365 days                | Rules that automatically <b>transition or delete objects</b> as they age, moving data to cheaper tiers or removing it once it's no longer needed                                                                            |
| Ephemeral Storage                | EC2 instance store volumes                                                   | <b>Temporary storage</b> physically attached to the host, high performance but data lost when the instance stops, suitable for temporary caches and buffers                                                                 |

TABLE 8  
Networking Concepts

The VPC is the foundation of cloud networking: your own isolated, software-defined network where you control IP ranges, subnets, and routing. The rest of the table builds outward from there: subnets and route tables segment and direct traffic, load balancers and CDNs distribute it, and gateways, peering, and connectivity options (VPN, Direct Connect, Transit Gateway, Private Link) govern how traffic flows in, out, and between networks securely.

| Concept                     | Example                                            | Description                                                                                                                 |
|-----------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Virtual Private Cloud (VPC) | Isolated network: 10.0.0.0/16 with private subnets | <b>Logically isolated network</b> within a cloud provider: full control over IP addressing, subnets, and route tables       |
| Subnet                      | Public subnet: 10.0.1.0/24 , Private: 10.0.2.0/24  | <b>Segment of a VPC's IP range</b> : resources in public subnets have internet access while private subnets remain isolated |

| Concept                                | Example                                                                          | Description                                                                                                                                                          |
|----------------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Route Table</b>                     | Route <code>0.0.0.0/0</code> to an internet gateway for a public subnet          | <b>Set of rules (routes)</b> that decide where traffic from a subnet or gateway goes, mapping a destination CIDR to a target such as a gateway or peering connection |
| <b>Load Balancer</b>                   | Distributing traffic across 10 web servers                                       | <b>Distributes incoming traffic</b> across multiple targets, improving availability and fault tolerance by routing around unhealthy instances                        |
| <b>Content Delivery Network (CDN)</b>  | CloudFront caching static assets at 400+ edge locations                          | <b>Globally distributed cache servers:</b> serve content closer to end users, reducing latency and bandwidth costs                                                   |
| <b>DNS Service</b>                     | Route 53 resolving <code>example.com</code> to load balancer IP                  | <b>Domain Name System management:</b> translates domain names to IP addresses with routing policies for failover, geolocation, and latency                           |
| <b>Public IP Address (Elastic IP)</b>  | Remapping a <code>52.x.x.x</code> Elastic IP to a replacement instance           | <b>Static public IPv4 address</b> attached to an instance or interface, keeping a stable public endpoint even after the underlying resource is replaced              |
| <b>NAT Gateway</b>                     | Private instances accessing internet for software updates                        | Enables instances in <b>private subnets to reach the internet:</b> outbound-only connectivity that prevents unsolicited inbound connections                          |
| <b>VPC Peering</b>                     | Peering <code>vpc-A</code> and <code>vpc-B</code> so instances share private IPs | <b>Direct network connection between two VPCs</b> , routing traffic over private IP addresses without a gateway or VPN                                               |
| <b>Virtual Private Network (VPN)</b>   | Encrypted tunnel between on-premises and cloud VPC                               | <b>Secure encrypted connection</b> over the public internet: connects on-premises networks to cloud resources with IPsec encryption                                  |
| <b>Direct Connect / ExpressRoute</b>   | Dedicated 10 Gbps fiber connection to AWS                                        | <b>Private dedicated network connection:</b> bypasses the internet for more reliable, lower-latency, higher-throughput access to cloud resources                     |
| <b>Transit Gateway</b>                 | Connecting 50 VPCs and on-premises networks through a hub                        | <b>Central hub</b> that connects multiple VPCs and on-premises networks, replacing complex peering relationships                                                     |
| <b>Private Link / Private Endpoint</b> | Accessing S3 from VPC without traversing public internet                         | <b>Private connectivity</b> to cloud services: keeps traffic within the provider's network without exposure to the public internet                                   |

TABLE 9

## Database Services

There's no universal database, so the cloud offers a managed engine for every data shape. Relational databases handle structured, transactional data with ACID guarantees; NoSQL trades that rigidity for flexible schemas and horizontal scale; and in-memory stores deliver microsecond latency for caching. The specialized rows, data warehouses, time-series, graph, and lakehouse, exist because analytics, sensor streams, relationship data, and mixed structured/unstructured workloads each query very differently.

| Type                              | Example                                                   | Description                                                                                                                                                                                                        |
|-----------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Relational Databases (SQL)</b> | Amazon RDS (MySQL, PostgreSQL), Azure SQL Database        | <ul style="list-style-type: none"> <li>• <b>SQL databases</b> managed by the provider</li> <li>• automated backups, patching, scaling, and high availability for ACID-compliant transactional workloads</li> </ul> |
| <b>NoSQL Databases</b>            | DynamoDB, Azure Cosmos DB, MongoDB Atlas                  | <ul style="list-style-type: none"> <li>• <b>Non-relational databases</b> for flexible schemas</li> <li>• supports key-value, document, column-family, and graph models with horizontal scaling</li> </ul>          |
| <b>In-Memory Databases</b>        | Redis, Memcached on Amazon ElastiCache                    | <ul style="list-style-type: none"> <li>• <b>RAM-based databases</b> for microsecond latency</li> <li>• ideal for caching, session management, real-time analytics, and leaderboards</li> </ul>                     |
| <b>Data Warehouses</b>            | Amazon Redshift, Azure Synapse Analytics, Google BigQuery | <ul style="list-style-type: none"> <li>• <b>Columnar storage</b> optimized for analytics</li> <li>• handles petabyte-scale data with fast query performance for business intelligence and reporting</li> </ul>     |
| <b>Time-Series Databases</b>      | Amazon Timestream, Azure Data Explorer                    | Specialized for <b>timestamped data</b> , efficiently storing and querying IoT sensor data, metrics, and events with time-based aggregations                                                                       |
| <b>Graph Databases</b>            | Amazon Neptune, Azure Cosmos DB (Gremlin API)             | Optimized for <b>relationships between data</b> , using nodes, edges, and properties for social networks, recommendation engines, and fraud detection                                                              |

| Type                         | Example                                                   | Description                                                                                                                                                                                                                                               |
|------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Data Lake / Lakehouse</b> | AWS Lake Formation, Databricks Lakehouse, Azure Data Lake | <ul style="list-style-type: none"> <li>Centralized repository storing <b>structured and unstructured data</b> at any scale</li> <li>lakehouse architecture combines data lake flexibility with warehouse performance for both analytics and ML</li> </ul> |

TABLE 10

## Security and Compliance

Security in the cloud starts with the shared responsibility model: understanding the line between what the provider secures and what stays on you is the single most important concept here. The remaining rows are the controls you own on your side of that line, covering identity, encryption, network filtering and continuous compliance checks.

| Concept                                             | Example                                                                                  | Description                                                                                                                                                                                                          |
|-----------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Shared Responsibility Model</b>                  | Provider secures infrastructure; customer secures data and config                        | Defines <b>who is responsible for what</b> : the provider manages security of the cloud (physical, network, hypervisor), the customer manages security <i>in</i> the cloud (data, IAM, encryption, app config)       |
| <b>Identity and Access Management (IAM)</b>         | Creating user with read-only S3 access policy                                            | <b>Controls who can access what</b> : manages authentication and authorization with users, groups, roles, and fine-grained permissions                                                                               |
| <b>Principle of Least Privilege</b>                 | <code>s3:GetObject</code> on one bucket instead of <code>AdministratorAccess</code>      | Grants only the permissions <b>a task actually needs</b> , and enforces separation of duties for every interaction with resources                                                                                    |
| <b>Role-Based Access Control (RBAC)</b>             | Assign the "Virtual Machine Contributor" role, scoped to one resource group              | Grants permissions through <b>roles instead of individual users</b> : a role assignment binds a security principal, a role definition, and a scope                                                                   |
| <b>Single Sign-On (SSO) and Identity Federation</b> | Sign in once through Okta, reach every linked AWS account                                | <ul style="list-style-type: none"> <li><b>SSO</b> lets a user sign in once and reach many applications</li> <li><b>federation</b> extends that trust from an external identity provider over SAML or OIDC</li> </ul> |
| <b>Multi-Factor Authentication (MFA)</b>            | Requiring password + authenticator app token                                             | <b>Additional authentication factor</b> beyond a password: requires something you know (password) and something you have (device) for critical operations                                                            |
| <b>Encryption at Rest</b>                           | S3 bucket with AES-256 server-side encryption                                            | Data <b>encrypted while stored</b> on disk, using provider-managed or customer-managed keys to protect it from unauthorized physical access                                                                          |
| <b>Encryption in Transit</b>                        | TLS 1.3 for HTTPS connections to cloud services                                          | Data <b>encrypted during transmission</b> : uses SSL/TLS protocols to protect data moving between clients and cloud services or between cloud services                                                               |
| <b>Security Groups / Firewalls</b>                  | <code>Allow port 443 from 0.0.0.0/0</code><br><code>Allow port 22 from 10.0.0.0/8</code> | <b>Virtual firewalls</b> controlling inbound and outbound traffic: stateful rules applied at the instance level, based on protocol, port, and source or destination                                                  |
| <b>Zero Trust Security</b>                          | Verifying every request regardless of network location                                   | <b>"Never trust, always verify"</b> security model: requires strict identity verification for every user and device, even inside the network perimeter                                                               |
| <b>Defense in Depth</b>                             | Firewall + IAM + encryption + monitoring, so no single control's failure exposes data    | Layers <b>multiple independent security controls</b> at every level, from the network edge to the application code                                                                                                   |
| <b>Key Management Service (KMS)</b>                 | Creating and rotating encryption keys automatically                                      | <b>Centralized encryption key management</b> : create, rotate, and control cryptographic keys used to encrypt data, with audit logging                                                                               |
| <b>Web Application Firewall (WAF)</b>               | Blocking SQL injection and XSS attacks                                                   | <b>Application-layer protection</b> : filters malicious traffic based on rules for common web exploits before it reaches applications                                                                                |
| <b>Cloud Security Posture Management (CSPM)</b>     | Continuously scan cloud accounts and flag an open S3 bucket                              | <ul style="list-style-type: none"> <li>Continuously <b>scans cloud accounts for misconfigurations</b></li> <li>scores overall security posture and guides remediation</li> </ul>                                     |

| Concept                          | Example                                | Description                                                                                                                                        |
|----------------------------------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compliance Certifications</b> | SOC 2, ISO 27001, HIPAA, PCI DSS, GDPR | <b>Third-party validated compliance:</b> providers maintain certifications for industry standards and regulations, simplifying customer compliance |

TABLE 11

## Pricing and Cloud Economics

How you pay for the same resource can swing the bill dramatically. On-demand pricing is the flexible default, while reserved instances and savings plans trade a one to three year commitment for deep discounts, and spot pricing cuts costs further on reclaimable spare capacity. Cloud economics goes beyond the pricing model: total cost of ownership, fixed versus variable costs and licensing strategy decide whether cloud beats on-premises at all.

| Concept                                                  | Example                                                                                      | Description                                                                                                                                                                                                                                  |
|----------------------------------------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pay-As-You-Go (On-Demand, Consumption-Based)</b>      | \$0.10/hour for compute, \$0.023/GB-month for storage                                        | Pay only for <b>resources actually consumed</b> , billed by the second, hour or GB with no upfront cost or long-term commitment                                                                                                              |
| <b>Reserved Instances / Committed Use</b>                | 1-year commitment for 40% discount, 3-year for 72%                                           | <b>Commit to usage</b> for a discounted rate: pay upfront or monthly for a one to three year term, saving up to 72% over on-demand pricing                                                                                                   |
| <b>Savings Plans</b>                                     | Commit to \$10/hour for 72% discount vs on-demand                                            | <b>Flexible pricing model:</b> commit to a consistent hourly spend for one to three years, applied automatically across eligible compute services                                                                                            |
| <b>Spot / Preemptible Pricing</b>                        | Bidding for unused capacity at up to 90% discount                                            | <ul style="list-style-type: none"> <li>• Purchase <b>spare capacity</b> at deep discounts</li> <li>• prices fluctuate with supply and demand</li> <li>• an instance can be reclaimed with a <b>two-minute interruption notice</b></li> </ul> |
| <b>Free Tier</b>                                         | \$100 credit at signup, up to \$200 over 6 months, plus 30+ services always free             | <ul style="list-style-type: none"> <li>• New accounts get sign-up credits for a <b>six month trial</b> on the Free plan</li> <li>• over 30 services stay <b>always free</b> within monthly usage limits</li> </ul>                           |
| <b>Data Transfer Pricing</b>                             | First 100 GB/month outbound to the internet free, then billed per GB                         | Charges for <b>data movement</b> : inbound is typically free, outbound to the internet is billed per GB beyond the free allowance, and inter-region transfer adds cost too                                                                   |
| <b>Pricing Calculator</b>                                | Model an EC2 + RDS + S3 architecture to get a monthly cost estimate before deploying         | A free tool that builds a <b>cost estimate</b> for a planned architecture before you build it, comparing instance types, terms and regions                                                                                                   |
| <b>Cost Management / FinOps</b>                          | Budget alerts when spending exceeds \$1,000/month                                            | <b>Track and optimize cloud spending</b> through cost visibility, budgets, forecasting and anomaly detection, bringing engineering, finance and business teams into one shared practice                                                      |
| <b>Total Cost of Ownership (TCO)</b>                     | Comparing 3-year on-premises server, power and staff costs against equivalent cloud spend    | Adds up every cost of running a workload, upfront and ongoing, to compare on-premises against cloud rather than sticker price alone                                                                                                          |
| <b>Fixed vs Variable Costs</b>                           | Buying servers upfront (fixed) vs paying only for compute hours used (variable)              | On-premises hardware cost is largely <b>fixed</b> regardless of use, while cloud usage is billed as a <b>variable</b> cost that scales with demand                                                                                           |
| <b>Rightsizing</b>                                       | Downsizing an idle m5.2xlarge to m5.large based on utilization metrics                       | Matching instance size to actual utilization to cut waste, using tools like AWS Compute Optimizer to flag idle or oversized resources                                                                                                        |
| <b>Bring Your Own License (BYOL) vs License Included</b> | Bring an existing Windows Server license (BYOL) instead of paying for a license-included AMI | <ul style="list-style-type: none"> <li>• <b>License included</b> bundles vendor licensing into the hourly instance rate</li> <li>• <b>BYOL</b> reuses licenses you already own, subject to the vendor's terms</li> </ul>                     |

| Concept                                           | Example                                                                                                        | Description                                                                                                      |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Cost Factors (Region, Size, Data Transfer)</b> | Same VM costs more in one region than another; egress to the internet costs more than transfer within a region | Resource type, size, <b>region</b> , subscription type and data transfer all change the bill for identical usage |

TABLE 12

## Resource Organization and Governance

How resources are grouped, billed and governed as an environment grows: the hierarchy of accounts, groups and resources, plus the guardrails, tags and locks that keep that hierarchy under control.

| Concept                                         | Example                                                                                                                       | Description                                                                                                                                                                                                                                           |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Account / Subscription / Project</b>         | AWS account<br>Azure subscription<br>GCP project                                                                              | The billing and isolation boundary: each <b>AWS account</b> , <b>Azure subscription</b> or <b>GCP project</b> holds its own resources, quotas and bill                                                                                                |
| <b>Resource Hierarchy</b>                       | AWS: Organization → OU → Account<br>Azure: Mgmt Group → Subscription → Resource Group<br>GCP: Organization → Folder → Project | <ul style="list-style-type: none"> <li>Nested levels that group resources for unified policy and access</li> <li>settings applied at a higher level are <b>inherited</b> by everything beneath it</li> </ul>                                          |
| <b>Organization / Management Group / Folder</b> | gcloud resource-manager folders create<br>--display-name=Engineering<br>--organization=ORG_ID                                 | A grouping layer above the billing boundary: AWS <b>organizational unit (OU)</b> , Azure <b>management group</b> , GCP <b>folder</b> , used to cascade policy and role assignments to many accounts at once                                           |
| <b>Resource Group</b>                           | az group create --name demoResourceGroup --location westus                                                                    | Azure's container below the subscription that groups resources sharing a lifecycle, for combined deployment, management and deletion                                                                                                                  |
| <b>Resource Tagging</b>                         | Environment: Production, Team: Engineering                                                                                    | <b>Key-value metadata</b> attached to resources: enables organization, cost allocation, access control and automation                                                                                                                                 |
| <b>Governance Policies</b>                      | Azure Policy: deny disallowed regions<br>AWS: Service Control Policy (SCP)<br>GCP: Organization Policy constraint             | Rules enforced across the hierarchy that block or flag non-compliant resources automatically, instead of relying on users to follow them                                                                                                              |
| <b>Resource Locks</b>                           | az lock create --name LockGroup --lock-type CanNotDelete --resource-group exempleresourcegroup                                | <ul style="list-style-type: none"> <li>Blocks accidental deletion or modification of a resource, resource group or subscription</li> <li>a <b>CanNotDelete</b> or <b>ReadOnly</b> lock applies to every user regardless of their RBAC role</li> </ul> |
| <b>Multi-Account Strategy</b>                   | Org: dev account, staging account, prod account                                                                               | Splitting workloads across separate accounts, subscriptions or projects for blast-radius isolation and clean billing, then unifying them under one organization                                                                                       |
| <b>Landing Zone</b>                             | AWS Control Tower setting up multi-account environment                                                                        | <b>Pre-configured multi-account environment</b> with guardrails: establishes security baselines, network architecture and governance policies for cloud adoption at scale                                                                             |

TABLE 13

## Management and Monitoring

Operating live workloads means reaching them through a console, CLI, SDK, or infrastructure as code, then watching them with monitoring, auto scaling, and service health tools. Backup and disaster recovery keep data safe, and SLAs define the uptime a provider promises.

| Concept                                          | Example                                                                   | Description                                                                                                                                                                                                       |
|--------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Management Console</b>                        | AWS Management Console, Azure Portal, Google Cloud Console                | Browser-based <b>graphical interface</b> for creating, reviewing, and monitoring resources without writing code                                                                                                   |
| <b>CLI and Cloud Shell</b>                       | <pre>aws s3 ls az vm list gcloud compute instances list</pre>             | <ul style="list-style-type: none"><li>• Command-line tools script and automate resource management</li><li>• a <b>pre-authenticated</b> browser-based cloud shell runs them with no local install</li></ul>       |
| <b>SDKs and APIs</b>                             | <pre>import boto3 ec2 = boto3.client("ec2") ec2.run_instances(...)</pre>  | Language-specific libraries and REST/HTTP APIs for <b>programmatic access</b> , used to build and automate against a provider's services                                                                          |
| <b>Infrastructure as Code (IaC)</b>              | Terraform, AWS CloudFormation, Pulumi, OpenTofu                           | Defines infrastructure in version-controlled configuration files for <b>repeatable and auditable</b> deployments                                                                                                  |
| <b>Monitoring and Observability</b>              | CloudWatch, Datadog, Grafana tracking CPU and latency                     | Collects and correlates <b>metrics, logs, and traces</b> , powering dashboards and alerting on resource utilization and application health                                                                        |
| <b>Auto Scaling</b>                              | Scale from 2 to 10 instances when CPU > 70%                               | Automatically adds or removes resources based on demand policies (target tracking, step, scheduled), keeping performance steady while controlling cost                                                            |
| <b>Service Health Dashboard</b>                  | AWS Health Dashboard, Azure Service Health, Google Cloud Status Dashboard | Ongoing visibility into service and resource availability, with alerts on outages and planned changes that could affect a workload                                                                                |
| <b>Advisor Recommendations (Trusted Advisor)</b> | AWS Trusted Advisor, Azure Advisor, Google Cloud Recommender              | Inspects an environment against best practices and recommends fixes to save money, improve performance, or close security gaps                                                                                    |
| <b>Backup and Disaster Recovery</b>              | Automated daily snapshots with 30-day retention                           | <ul style="list-style-type: none"><li>• Scheduled backups with point-in-time recovery protect data</li><li>• cross-region replication and DR runbooks (RTO/RPO targets) restore service after a failure</li></ul> |
| <b>Service Level Agreements (SLAs)</b>           | EC2 SLA: 99.99% monthly uptime (~4.3 min downtime/month)                  | Contractual uptime guarantees from a provider, defining availability targets (e.g., three nines = 99.9%, four nines = 99.99%) and credits for breaches                                                            |

TABLE 14

## Development and DevOps

These are the building blocks of modern cloud-native applications and the pipelines that ship them. CI/CD automates build-test-deploy, container orchestration manages fleets of containers, and API gateways expose and protect your services. Message queues and event streaming decouple components for resilience, while artifact registries, secrets management, and service mesh handle storing builds, guarding credentials, and securing service-to-service traffic.

| Concept                        | Example                                                   | Description                                                                                                                       |
|--------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>CI/CD Pipelines</b>         | GitHub Actions, GitLab CI, AWS CodePipeline, Azure DevOps | Automates build, test, and deployment, validating and shipping code changes continuously                                          |
| <b>Container Orchestration</b> | Kubernetes managing 100 containerized microservices       | <b>Automated container lifecycle management:</b> scheduling, scaling, networking, and self-healing for containerized applications |
| <b>API Gateway</b>             | Exposing Lambda functions as REST/HTTP APIs               | <b>Managed API creation and management:</b> publishes and secures APIs at scale with authentication, throttling, and monitoring   |
| <b>Message Queues</b>          | Amazon SQS, Azure Service Bus, Google Pub/Sub             | <b>Asynchronous message passing</b> that decouples components using queues for reliable delivery between distributed systems      |

| Concept                   | Example                                                        | Description                                                                                                                                                        |
|---------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Event Streaming</b>    | Amazon MSK (Kafka), Amazon Kinesis, Azure Event Hubs           | <b>Real-time data streaming</b> for high-throughput event pipelines used in analytics, log aggregation, and event-driven architectures                             |
| <b>Artifact Registry</b>  | Amazon ECR, Azure Container Registry, Google Artifact Registry | Stores and manages build artifacts: private registries for container images, packages, and dependencies with vulnerability scanning                                |
| <b>Secrets Management</b> | Storing database credentials with automatic rotation           | <b>Centralized secrets storage</b> that securely stores and rotates credentials, API keys, and certificates with encryption and access control                     |
| <b>Service Mesh</b>       | Istio, Linkerd, Consul Connect on Kubernetes                   | A dedicated infrastructure layer for service-to-service communication, providing traffic management, observability, and security ( <b>mTLS</b> ) for microservices |

TABLE 15

## Cloud Adoption and Migration

When moving existing workloads to the cloud, the "7 Rs" are the standard menu of approaches, ranging from minimal change to total reinvention: rehosting is the fast lift-and-shift that changes nothing, replatforming makes small cloud optimizations along the way, and refactoring fully re-architects for cloud-native benefits, while the rest cover the choices to repurchase a SaaS alternative, relocate at the hypervisor level, retire what's no longer needed, or simply retain it on-premises for now. A Cloud Adoption Framework and dedicated migration tooling guide the plan behind those choices.

| Strategy                                | Example                                                                                                                                                          | Description                                                                                                                                                                                                                                                  |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Rehost (Lift and Shift)</b>          | Moving VM from VMware to AWS EC2 unchanged                                                                                                                       | <b>Migrates without modification</b> , moving applications as-is to cloud infrastructure: the fastest approach, but it skips cloud-native benefits                                                                                                           |
| <b>Replatform (Lift and Reshape)</b>    | Migrating database from self-managed MySQL to Amazon RDS                                                                                                         | <b>Makes targeted cloud optimizations</b> during migration, such as switching to a managed service, without changing the core architecture                                                                                                                   |
| <b>Refactor (Re-architect)</b>          | Redesigning monolith as microservices with containers                                                                                                            | <b>Re-architects for cloud-native use</b> , rebuilding the application to use features like auto scaling, serverless and managed services                                                                                                                    |
| <b>Repurchase (Drop and Shop)</b>       | Replacing on-premises CRM with Salesforce SaaS                                                                                                                   | <b>Switches to a different product</b> , typically moving from licensed software to a SaaS alternative                                                                                                                                                       |
| <b>Relocate (Hypervisor-Level Lift)</b> | VMware Cloud on AWS moving entire vSphere environment                                                                                                            | <b>Moves VMs at the infrastructure level</b> without buying new hardware, using hypervisor compatibility between source and cloud                                                                                                                            |
| <b>Retire</b>                           | Decommissioning unused legacy application                                                                                                                        | <b>Turns off applications no longer needed</b> , shrinking the scope and cost of the migration                                                                                                                                                               |
| <b>Retain</b>                           | Keeping mainframe system on-premises for now                                                                                                                     | <b>Keeps the application in its current environment</b> , deferring migration until it is ready or required to stay on-premises                                                                                                                              |
| <b>Cloud Adoption Framework</b>         | AWS CAF: Business, People, Governance, Platform, Security, Operations perspectives<br>Microsoft CAF: Strategy, Plan, Ready, Adopt, Govern, Secure, Manage phases | <ul style="list-style-type: none"> <li>• Vendor-published best practices, tools and guidance for planning and governing a cloud adoption</li> <li>• <b>AWS CAF</b> groups them into six perspectives, <b>Microsoft's CAF</b> into adoption phases</li> </ul> |
| <b>Database Migration Service</b>       | Replicating an on-premises Oracle database to Amazon RDS with the source staying online                                                                          | <b>Migrates and replicates databases</b> to the cloud with minimal downtime, keeping the source system operational until cutover                                                                                                                             |
| <b>Offline Data Transfer Device</b>     | Shipping an Azure Data Box device to move 200 TB of on-premises backups into cloud storage                                                                       | Ships a physical storage device to move very large datasets when the network link is too slow or too costly for an online transfer                                                                                                                           |

TABLE 16

## Well-Architected Framework Pillars

These six pillars are the lens cloud providers use to evaluate whether a workload is built well, and they make a useful checklist for any architecture review. Each one names a tension worth balancing: operational excellence and reliability for running smoothly and recovering from failure, security for protecting it, performance efficiency and cost optimization for doing more with less, and sustainability for minimizing environmental impact.

| Pillar                         | Example                                                      | Description                                                                                                                                                          |
|--------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Operational Excellence</b>  | Automated runbooks, IaC deployments, incident response plans | Run and monitor systems to <b>deliver business value</b> : automate operations, respond to events, and continuously improve processes                                |
| <b>Security (Pillar)</b>       | Least-privilege IAM, encryption everywhere, audit logging    | <b>Protect information and systems</b> : implement strong identity controls, enable traceability, apply security at all layers, and automate security best practices |
| <b>Reliability (Pillar)</b>    | Multi-AZ deployments, auto-recovery, chaos engineering       | Ensure workloads perform <b>correctly and consistently</b> : design for automatic recovery from failure, test disaster procedures, and scale horizontally            |
| <b>Performance Efficiency</b>  | Right-sizing instances, using serverless, caching layers     | Use computing resources <b>efficiently as demand changes</b> : select the right resource types, monitor performance, and evolve with new technologies                |
| <b>Cost Optimization</b>       | Reserved instances, spot usage, rightsizing, idle shutdown   | Deliver value at the <b>lowest price point</b> : understand spending, select the right pricing models, and eliminate waste through continuous optimization           |
| <b>Sustainability (Pillar)</b> | Choosing efficient instance types, reducing idle resources   | Minimize <b>environmental impact</b> of cloud workloads: select efficient hardware, right-size resources, and use managed services to share infrastructure           |

## References

### Official Documentation

1. Infrastructure as a Service (IaaS)
2. Software as a Service (SaaS)
3. Function as a Service (FaaS / Serverless)
4. Hybrid Cloud
5. Cost Savings (CapEx to OpEx)
6. Global Reach
7. High Availability
8. Security and Governance
9. Predictability (Performance and Cost)
10. Manageability
11. Region
12. Availability Zone (AZ)
13. Edge Location
14. Local Zone
15. Region Pair
16. Government / Sovereign Region
17. Data Sovereignty
18. Containers
19. Hypervisor
20. Bare Metal Instances
21. GPU Instances
22. Object Storage
23. Block Storage
24. File Storage
25. Storage Tiers
26. Archive Storage
27. Storage Redundancy / Replication
28. Lifecycle Policies
29. Ephemeral Storage
30. Virtual Private Cloud (VPC)
31. Subnet
32. Route Table
33. Load Balancer
34. Content Delivery Network (CDN)
35. DNS Service
36. Public IP Address (Elastic IP)
37. NAT Gateway
38. VPC Peering
39. Virtual Private Network (VPN)
40. Direct Connect / ExpressRoute
41. Transit Gateway
42. Private Link / Private Endpoint
43. Relational Databases (SQL)
44. NoSQL Databases
45. In-Memory Databases
46. Data Warehouses
47. Time-Series Databases
48. Graph Databases
49. Data Lake / Lakehouse
50. Shared Responsibility Model
51. Identity and Access Management (IAM)
52. Principle of Least Privilege
53. Role-Based Access Control (RBAC)
54. Single Sign-On (SSO) and Identity Federation
55. Multi-Factor Authentication (MFA)
56. Encryption at Rest
57. Encryption in Transit
58. Security Groups / Firewalls

59. Key Management Service (KMS)
61. Cloud Security Posture Management (CSPM)
63. Pay-As-You-Go (On-Demand, Consumption-Based)
65. Savings Plans
67. Free Tier
69. Pricing Calculator
71. Fixed vs Variable Costs
73. Bring Your Own License (BYOL) vs License Included
75. Account / Subscription / Project
77. Organization / Management Group / Folder
79. Resource Tagging
81. Resource Locks
83. Landing Zone
85. CLI and Cloud Shell
87. Infrastructure as Code (IaC)
89. Auto Scaling
91. Advisor Recommendations (Trusted Advisor)
93. Service Level Agreements (SLAs)
95. Container Orchestration
97. Message Queues
99. Artifact Registry
101. Service Mesh
103. Cloud Adoption Framework
105. Offline Data Transfer Device
107. Security (Pillar)
109. Performance Efficiency
111. Sustainability (Pillar)
113. Spot / Preemptible Pricing
115. AWS - Amazon ECS <https://aws.amazon.com/ecs/>
117. AWS - Types of Cloud Computing <https://aws.amazon.com/types-of-cloud-computing/>
119. AWS Lambda <https://aws.amazon.com/lambda/>
121. AWS CloudFormation <https://aws.amazon.com/cloudformation/>
123. Kubernetes Concepts <https://kubernetes.io/docs/concepts/overview/>
125. AWS Well-Architected Whitepaper <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>
127. AWS Organizations <https://aws.amazon.com/organizations/>
129. Azure Architecture Center <https://learn.microsoft.com/en-us/azure/architecture/>
131. AWS Compute Blog <https://aws.amazon.com/blogs/compute/>
133. Microsoft Learn for Azure <https://learn.microsoft.com/en-us/training/azure/>
135. OpenTelemetry Observability Primer <https://opentelemetry.io/docs/concepts/observability-primer/>
137. Azure Architecture Best Practices <https://learn.microsoft.com/en-us/azure/architecture/best-practices/>
139. AWS Security Best Practices <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
141. AWS Cost Optimization Best Practices <https://aws.amazon.com/aws-cost-management/aws-cost-optimization/>
143. Docker Documentation <https://docs.docker.com/>
145. OpenStack Documentation <https://docs.openstack.org/>
147. AWS Amplify <https://aws.amazon.com/amplify/>
149. AWS - Compare Block, File, Object Storage <https://aws.amazon.com/compare/the-difference-between-block-file-object-storage/>
151. AWS Disaster Recovery Shared Responsibility <https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/shared-responsibility.html>
60. Web Application Firewall (WAF)
62. Compliance Certifications
64. Reserved Instances / Committed Use
66. Spot / Preemptible Pricing
68. Data Transfer Pricing
70. Total Cost of Ownership (TCO)
72. Rightsizing
74. Cost Factors (Region, Size, Data Transfer)
76. Resource Hierarchy
78. Resource Group
80. Governance Policies
82. Multi-Account Strategy
84. Management Console
86. SDKs and APIs
88. Monitoring and Observability
90. Service Health Dashboard
92. Backup and Disaster Recovery
94. CI/CD Pipelines
96. API Gateway
98. Event Streaming
100. Secrets Management
102. Rehost (Lift and Shift)
104. Database Migration Service
106. Operational Excellence
108. Reliability (Pillar)
110. Cost Optimization
112. Virtual Machines
114. Cloud Adoption Framework
116. AWS - Machine Learning <https://aws.amazon.com/machine-learning/>
118. AWS - Amazon EC2 <https://aws.amazon.com/ec2/>
120. AWS Security <https://aws.amazon.com/security/>
122. AWS Backup <https://aws.amazon.com/backup/>
124. AWS Well-Architected Framework <https://aws.amazon.com/architecture/well-architected/>
126. Microsoft Azure Cloud Adoption Framework <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/considerations/fundamental-concepts>
128. AWS Architecture Blog <https://aws.amazon.com/blogs/architecture/>
130. AWS News Blog <https://aws.amazon.com/blogs/aws/>
132. AWS Training and Certification <https://aws.amazon.com/training/>
134. AWS Best Practices for Event-Driven Architectures <https://aws.amazon.com/blogs/architecture/best-practices-for-implementing-event-driven-architectures-in-your-organization/>
136. AWS Cost Management <https://aws.amazon.com/aws-cost-management/>
138. Google Cloud Best Practices <https://cloud.google.com/docs/enterprise/best-practices-for-enterprise-organizations>
140. Microsoft Cloud Adoption Framework <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/>
142. Azure Well-Architected Framework <https://learn.microsoft.com/en-us/azure/well-architected/>
144. Azure Resource Manager Templates <https://learn.microsoft.com/en-us/azure/azure-resource-manager/templates/>
146. AWS Outposts <https://aws.amazon.com/outposts/>
148. AWS Global Infrastructure <https://aws.amazon.com/about-aws/global-infrastructure/>
150. AWS Networking Services Whitepaper <https://docs.aws.amazon.com/whitepapers/latest/aws-overview/networking-services.html>
152. Google Compute Engine - Bare Metal Instances <https://docs.cloud.google.com/compute/docs/instances/bare-metal-instances>

153. Pulumi Documentation <https://www.pulumi.com/docs/>

154. AWS Elastic Disaster Recovery  
<https://aws.amazon.com/disaster-recovery/>

## Specifications & Standards

155. On-Demand Self-Service

156. Community Cloud

157. Zero Trust Security

158. NIST Definition of Cloud Computing (SP 800-145) <https://csrc.nist.gov/pubs/sp/800/145/final>

159. NIST Zero Trust Architecture <https://www.nist.gov/publications/zero-trust-architecture>

160. NIST Cloud Computing Reference Architecture (SP 500-292)  
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication500-292.pdf>

## Source Code & Repositories

161. AWS Samples GitHub <https://github.com/aws-samples>

163. Google Cloud Platform GitHub <https://github.com/GoogleCloudPlatform>

165. AWS CDK Examples <https://github.com/aws-samples/aws-cdk-examples>

167. Kubernetes Examples <https://github.com/kubernetes/examples>

169. Azure SDK for Python <https://github.com/Azure/azure-sdk-for-python>

171. Pulumi Examples <https://github.com/pulumi/examples>

162. Azure Samples GitHub <https://github.com/Azure-Samples>

164. Terraform AWS Modules <https://github.com/terraform-aws-modules>

166. Azure Quickstart Templates  
<https://github.com/Azure/azure-quickstart-templates>

168. AWS SDK for Python (Boto3) <https://github.com/boto/boto3>

170. Google Cloud Python Client  
<https://github.com/googleapis/google-cloud-python>

172. OpenTofu <https://github.com/opentofu/opentofu>

## Articles & Tutorials

173. Platform as a Service (PaaS)

175. AI as a Service (AIaaS)

177. Container as a Service (CaaS)

179. Disaster Recovery as a Service (DRaaS)

181. Private Cloud

183. Sovereign Cloud

185. Virtual Machines

187. Microsoft Azure - What is Cloud Computing? <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-cloud-computing/>

189. Firebase Documentation <https://firebase.google.com/docs>

191. Microsoft Azure - What are IaaS, PaaS, and SaaS? <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-are-iaas-paas-and-saas>

193. Terraform Documentation <https://www.terraform.io/docs>

195. Splunk - Data Sovereignty vs Data Residency <https://www.splunk.com/en-us/blog/learn/data-sovereignty-vs-data-residency.html>

197. Google Cloud Blog <https://cloud.google.com/blog/>

199. Google Cloud Release Notes <https://cloud.google.com/release-notes>

201. Splunk - Availability Zones Guide  
[https://www.splunk.com/en\\_us/blog/learn/availability-zones.html](https://www.splunk.com/en_us/blog/learn/availability-zones.html)

203. AWS re:Invent Conference <https://reinvent.awsevents.com/>

205. Google Cloud Next <https://cloud.withgoogle.com/next>

207. Wiz - Cloud Security Best Practices  
<https://www.wiz.io/academy/cloud-security/cloud-security-best-practices>

209. IBM - 7 Rs of Cloud Migration  
<https://www.ibm.com/think/insights/7-rs-cloud-migration>

211. Cloudflare - Vendor Lock-In  
<https://www.cloudflare.com/learning/cloud/what-is-vendor-lock-in/>

213. CNCF Cloud Native Glossary <https://glossary.cncf.io/>

215. Cloud Security Alliance <https://cloudsecurityalliance.org/>

217. Microsoft Trust Center <https://www.microsoft.com/en-us/trust-center>

219. Google Cloud Architecture Framework  
<https://cloud.google.com/architecture/framework>

221. edX - Cloud Computing Courses  
<https://www.edx.org/learn/cloud-computing>

174. Database as a Service (DBaaS)

176. Backend as a Service (BaaS)

178. Network as a Service (NaaS)

180. Public Cloud

182. Multi-Cloud

184. Scalability

186. Cost Management / FinOps

188. Google Cloud - What is Cloud Computing?  
<https://cloud.google.com/learn/what-is-cloud-computing>

190. Google Cloud - Sovereign Controls  
<https://cloud.google.com/sovereign-controls>

192. IBM - IaaS, PaaS, SaaS  
<https://www.ibm.com/think/topics/iaas-paas-saas>

194. FinOps Foundation <https://www.finops.org/>

196. Amazon Sustainability  
<https://sustainability.aboutamazon.com/products-services/the-cloud>

198. Azure Updates <https://azure.microsoft.com/en-us/updates/>

200. IBM - What is XaaS? <https://www.ibm.com/think/topics/xaas>

202. Synopsys - Essential Cloud Computing Characteristics <https://www.synopsys.com/blogs/chip-design/essential-cloud-computing-characteristics.html>

204. Microsoft Ignite <https://ignite.microsoft.com/>

206. Google Cloud Skills Boost <https://www.cloudskillsboost.google/>

208. Wiz - Shared Responsibility Model  
<https://www.wiz.io/academy/cloud-security/shared-responsibility-model>

210. The Twelve-Factor App Methodology <https://12factor.net/>

212. VMware - DRaaS  
<https://www.vmware.com/topics/disaster-recovery-service-draas>

214. Google Cloud Adoption Framework  
<https://cloud.google.com/adoption-framework>

216. Cloud Native Computing Foundation <https://www.cncf.io/>

218. Google Cloud Trust Center <https://cloud.google.com/security>

220. Coursera - Cloud Computing Specializations  
<https://www.coursera.org/specializations/cloud-computing>

222. A Cloud Guru <https://acloudguru.com/>

- 223.** Cloud Academy <https://cloudacademy.com/>
- 225.** Google Kubernetes Engine  
<https://cloud.google.com/kubernetes-engine/docs>
- 227.** VMware Cloud  
<https://www.vmware.com/products/cloud-infrastructure.html>
- 229.** Azure Arc <https://azure.microsoft.com/en-us/products/azure-arc/>
- 231.** Azure Government  
<https://azure.microsoft.com/en-us/explore/global-infrastructure/government/>
- 233.** Google BigQuery <https://cloud.google.com/bigquery/docs>
  
- 235.** Google Cloud SDK <https://cloud.google.com/sdk/docs>
  
- 237.** IBM - Object vs File vs Block Storage  
<https://www.ibm.com/think/topics/object-vs-file-vs-block-storage>
- 239.** Azure - What are Private, Public, Hybrid Clouds? <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-are-private-public-hybrid-clouds>
  
- 224.** Pluralsight Cloud Courses <https://www.pluralsight.com/cloud>
- 226.** Azure Container Instances  
<https://azure.microsoft.com/en-us/products/container-instances/>
- 228.** Azure Stack <https://azure.microsoft.com/en-us/products/azure-stack/>
  
- 230.** Google Anthos <https://cloud.google.com/anthos/docs>
- 232.** Microsoft Azure SQL Database  
<https://azure.microsoft.com/en-us/products/azure-sql/database/>
- 234.** Microsoft Azure Synapse Analytics  
<https://azure.microsoft.com/en-us/products/synapse-analytics/>
- 236.** Google Cloud - Object vs Block vs File Storage  
<https://cloud.google.com/discover/object-vs-block-vs-file-storage>
- 238.** CNCF State of Cloud Native Development Q1 2026  
<https://www.cncf.io/reports/state-of-cloud-native-development-q1-2026/>

## Video

- 240.** AWS Online Tech Talks <https://www.youtube.com/c/AWSOnlineTechTalks>
  
- 242.** Google Cloud Tech YouTube  
<https://www.youtube.com/c/GoogleCloudTech>
- 244.** Cloud Native Computing Foundation YouTube  
<https://www.youtube.com/c/CloudNativeComputingFoundation>
- 246.** The Ultimate Beginner's Guide to Cloud Computing in 2026  
<https://www.youtube.com/watch?v=AOUSI1b7DAw>
  
- 241.** Microsoft Azure YouTube Channel  
<https://www.youtube.com/c/MicrosoftAzure>
- 243.** AWS re:Invent Videos  
<https://www.youtube.com/user/AmazonWebServices>
- 245.** freeCodeCamp - AWS Certified Cloud Practitioner  
<https://www.youtube.com/watch?v=SOTamWNgDKc>